```
while ($token !== false) {
    print "{$token}\n";
    $index = !$index % 2;
    $token = strtok($delims[$index]);
}
```

```
Output:
foo
bar
cat
miaow
dog
bark
```

Here we are alternating between two delimiters and parsing a URL query string (the evil-looking expression `!$index % 2` simply flip-flops between `0` and `1` on each loop iteration). Why didn't we just use `'=&` as a single delimiter value? Here's why...

```
$query = 'foo=b=ar&ca&t=miaow&dog=bark';
// rest of code as before...
```

```
Output:
foo
b=ar
ca&t
miaow
dog
bark
```

If we simply defined the delimiters as `'&='` on every call, the tokens would be `foo`, `b`, `ar`, `ca`, and so on. We can change our delimiters on each token fetch to suit whatever framework of rules we wish to tokenize with. With that said, this is *not* the way I recommend that query strings be parsed! There is the core function `parse_str()` for that task, which is featured in Chapter 11, "Programming for the Internet."

One final word of caution: `strtok()`, as you can probably guess, is not multi-byte safe. If you need to tokenize multi-byte strings, the safest way to do that in PHP is to use one of the regular expression functions, and we'll be meeting those later in the chapter.